

**GROUP 6**

**CLOUD SECURITY BEST PRACTICES**

**SECURITY ASSESSMENT AND HARDENING OF A SMALL AWS**

**CLOUD ENVIRONMENT**

**PROJECT REPORT**

Submitted in partial fulfilment of the requirements for the SkillUp Imo Cloud Engineering Study

Session

**AWS Cloud Security Best Practices**

**GROUP MEMBERS**

| <b>S/N</b> | <b>Name</b>                  | <b>Registration Number</b> |
|------------|------------------------------|----------------------------|
| 1          | NKWUOH CHUKWUEBUKA W.        | SKIMMMC-402033             |
| 2          | NWANKWO CHIAGHANAM           | SKIMMMC-402341             |
| 3          | OGBONNA CHURCHILL AKPODIKUMO | SKIMMMC-402042             |
| 4          | OKONKWO ANTHONY CHUKWUEMEKA  | SKIMMMC-402053             |
| 5          | OKORO SIXTUS CHIJOKE         | SKIMMMC-402027             |

Date: 21/07/2026

## EXECUTIVE SUMMARY

This project presents a security assessment and hardening exercise conducted on a small Amazon Web Services (AWS) cloud environment. The project was designed to demonstrate practical understanding of cloud security best practices through the review of network security controls, identity and access management, Amazon EC2 key pair management, and Amazon S3 permissions. The assessment follows a professional security lifecycle of Discover, Assess, Remediate, Validate, and Report.

The implementation approach uses AWS Command Line Interface (AWS CLI) and Bash scripting to automate infrastructure creation and security review activities. The environment is designed around an Amazon Virtual Private Cloud (VPC), public subnet, Security Group, EC2 instance, IAM identities, and an S3 bucket. The project also emphasizes secure documentation practices by storing scripts, reports, screenshots, and project evidence in a GitHub repository while ensuring that sensitive credentials and private keys are excluded through a .gitignore file.

The expected outcome is a documented security assessment that identifies weaknesses such as unrestricted administrative access, excessive IAM permissions, insecure key management, and potentially unsafe S3 access. The project demonstrates how such findings can be documented, assigned risk levels, remediated, and validated using evidence. The report concludes with recommendations for least privilege, multi-factor authentication, restricted network access, encryption, secure credential management, logging, monitoring, and continuous security reviews.

## TABLE OF CONTENTS

1. Introduction
  2. Problem Statement
  3. Aim and Objectives
  4. Scope of the Project
  5. AWS Services Used
  6. Tools Used
  7. Methodology
  8. Project Environment and Architecture
  9. Implementation Steps
  10. Security Assessment and Findings
  11. Security Remediation and Validation
  12. GitHub Documentation and Version Control
  13. Challenges Encountered
  14. Lessons Learned
  15. Security Recommendations
  16. Live Demonstration Plan
  17. Conclusion
  18. References
- Appendix A: Project Repository Structure
- Appendix B: Screenshot Placeholders

## 1. INTRODUCTION

Cloud computing has transformed the way organisations deploy applications, store information, and provide digital services. AWS provides a broad range of infrastructure and platform services that allow users to create scalable environments without owning physical data centres. However, the flexibility of cloud platforms also introduces security responsibilities. Misconfigured network controls, excessive identity permissions, exposed storage, and poor credential management can create vulnerabilities that may be exploited by unauthorised individuals.

This project focuses on Cloud Security Best Practices through a practical review of a small AWS environment. The project combines infrastructure automation, security assessment, remediation, documentation, and presentation. The assessment is structured around four major areas: Security Groups, IAM, EC2 key pair management, and S3 permissions. These areas were selected because they represent common controls that directly affect the confidentiality, integrity, and availability of cloud resources.

The project also introduces an evidence-driven security process. Rather than simply stating that security controls exist, the team is expected to inspect configurations, identify weaknesses, record findings, implement appropriate improvements, and validate the changes. Screenshots and command outputs provide supporting evidence for the final report.

## 2. PROBLEM STATEMENT

Small cloud environments are frequently deployed quickly for learning, development, testing, or early-stage applications. During rapid deployment, security controls may be configured incorrectly or may not be reviewed after the initial deployment. Examples include allowing SSH access from any Internet address, assigning excessive IAM permissions, storing private keys insecurely, or failing to restrict public access to S3 buckets.

These weaknesses can increase the likelihood of unauthorised access, data exposure, credential compromise, or accidental modification of cloud resources. The problem addressed by this project is therefore the need to systematically review a small AWS environment, identify security gaps, apply practical controls, and produce an auditable security report.

### **3. AIM AND OBJECTIVES**

The primary aim of the project is to assess and improve the security posture of a small AWS cloud environment using practical cloud security best practices.

The specific objectives are to:

- Review AWS Security Group configurations and identify overly permissive inbound rules.
- Review IAM users, groups, roles, and policies with emphasis on the principle of least privilege.
- Discuss and demonstrate secure EC2 key pair management.
- Review Amazon S3 permissions and public access controls.
- Identify, document, and classify security findings according to risk.
- Apply selected remediation actions and validate the resulting configuration.
- Use AWS CLI and Bash scripting to automate infrastructure and audit activities.
- Store project scripts, documentation, screenshots, and reports in a GitHub repository.
- Present security recommendations and demonstrate the project workflow to an audience.

### **4. SCOPE OF THE PROJECT**

The project scope covers a small AWS environment consisting of an Amazon VPC, a public subnet, an Internet Gateway, a route table, an EC2 instance, a Security Group, IAM identities

and policies, an S3 bucket, and associated security controls. The scope includes configuration review and controlled remediation.

The project does not attempt to perform penetration testing, exploit vulnerabilities, or conduct unauthorised security testing. All assessment activities are limited to resources owned or explicitly authorised for the project. The project is intended as an educational security assessment and demonstration of defensive cloud engineering practices.

## 5. AWS SERVICES USED

| AWS Service     | Purpose in the Project                                                          |
|-----------------|---------------------------------------------------------------------------------|
| Amazon VPC      | Provides an isolated virtual network for the project environment.               |
| Amazon EC2      | Provides a virtual machine that represents a small cloud workload.              |
| Security Groups | Control network traffic permitted to and from EC2 resources.                    |
| AWS IAM         | Manages identities, groups, roles, policies, and access permissions.            |
| Amazon S3       | Provides cloud object storage for project-related documentation or test data.   |
| AWS CloudTrail  | Recommended for recording AWS API activity and supporting security auditing.    |
| AWS CLI         | Provides command-line access for resource creation, inspection, and automation. |

## 6. TOOLS USED

- AWS CLI for infrastructure management and security inspection.
- Bash scripting for automation.
- Git for source control.
- GitHub for repository hosting and documentation.
- Visual Studio Code for editing scripts and project files.
- AWS Management Console for visual verification and screenshots.
- Draw.io or a similar diagramming tool for the architecture diagram.

## 7. METHODOLOGY

The project follows a five-stage security assessment lifecycle: Discover, Assess, Remediate, Validate, and Report. The Discover stage involves identifying the AWS account, resources, network components, identities, and storage resources. The Assess stage examines configurations for security weaknesses. The Remediate stage applies appropriate security improvements. The Validate stage confirms that remediation has been successful. The Report stage records findings, evidence, recommendations, and lessons learned.

| Stage    | Activity                                        | Expected Evidence                 |
|----------|-------------------------------------------------|-----------------------------------|
| Discover | Inventory AWS resources and configurations.     | AWS CLI outputs and screenshots.  |
| Assess   | Review Security Groups, IAM, key pairs, and S3. | Audit outputs and findings table. |

|           |                                                |                                           |
|-----------|------------------------------------------------|-------------------------------------------|
| Remediate | Correct selected security weaknesses.          | Before/after screenshots.                 |
| Validate  | Confirm that controls are working as intended. | Validation commands and console evidence. |
| Report    | Document results and recommendations.          | Final report and GitHub repository.       |

## 8. PROJECT ENVIRONMENT AND ARCHITECTURE

The proposed environment uses an AWS VPC with CIDR block 10.0.0.0/16. A public subnet with CIDR block 10.0.1.0/24 is used for the demonstration EC2 workload. An Internet Gateway provides Internet connectivity, while a route table directs Internet-bound traffic from the public subnet. A Security Group controls inbound and outbound traffic.

IAM provides identity and access management for administrative and operational activities. Amazon S3 is used as a storage service whose public access, encryption, and versioning settings are reviewed. The architecture is designed to demonstrate how security controls operate across network, identity, compute, and storage layers.

Figure 1: AWS Cloud Security Architecture (Insert the final architecture diagram here).

## 9. IMPLEMENTATION STEPS

### 9.1 AWS CLI Authentication

The AWS CLI was configured with an authorised AWS identity and the project region was set to Europe (London), represented by the region code eu-west-2. The identity was validated using the



AWS Security Token Service command `aws sts get-caller-identity`. This step confirmed that the CLI was authenticated before resource creation.

Evidence placeholder: Insert Figure 2 – AWS CLI authentication output.

## 9.2 VPC Creation

A VPC named `cloud-security-vpc` was created using the CIDR block `10.0.0.0/16`. The VPC was created through a Bash script using the AWS CLI. The script captured the resulting VPC ID and applied a Name tag for easy identification.

Evidence placeholder: Insert Figure 3 – VPC creation command output and AWS Console view.

## 9.3 Public Subnet and Network Connectivity

The next infrastructure stage involves creating a public subnet with CIDR block `10.0.1.0/24` within the VPC. An Internet Gateway and route table are associated with the VPC to provide controlled Internet connectivity for the demonstration workload. The route table should contain a default route to the Internet Gateway.

Evidence placeholder: Insert Figure 4 – VPC, subnet, Internet Gateway, and route table.

## 9.4 Security Group Configuration

A Security Group is configured to control traffic to the EC2 instance. For the controlled security demonstration, an intentionally permissive SSH rule may be created temporarily, subject to the team's authorised environment and project requirements. The initial configuration is documented as a security finding, after which SSH access is restricted to a trusted IP address using a `/32` CIDR range. HTTP and HTTPS should only be opened when required by the demonstration workload.

Evidence placeholder: Insert Figure 5 – Security Group before remediation.

Evidence placeholder: Insert Figure 6 – Security Group after remediation.

## 9.5 EC2 Deployment

An EC2 Linux instance is launched into the public subnet using an appropriate instance type eligible for the project's available AWS Free Tier or educational allocation. The instance is associated with the reviewed Security Group and an EC2 key pair. The team validates connectivity without exposing private key material.

Evidence placeholder: Insert Figure 7 – EC2 instance running.

## 9.6 IAM Review

The IAM review examines users, groups, roles, policies, and access keys. The assessment considers whether permissions are excessive, whether MFA is enabled for privileged users, whether long-lived access keys are necessary, and whether EC2 workloads use IAM roles instead of embedded access keys.

Evidence placeholder: Insert Figure 8 – IAM users, groups, or policies.

## 9.7 S3 Review

The S3 review examines bucket public access settings, bucket policies, encryption, and versioning. The recommended baseline is to enable S3 Block Public Access unless public access is explicitly required and approved. Server-side encryption and versioning are also recommended for appropriate workloads.

Evidence placeholder: Insert Figure 9 – S3 security configuration.

## 10. SECURITY ASSESSMENT AND FINDINGS

The assessment findings should be recorded in a structured findings register. Each finding should contain an identifier, category, description, risk level, recommendation, and status. The following table represents the recommended format and should be updated to reflect the actual findings discovered in the team's AWS environment.

| ID      | Area           | Finding                                                           | Risk | Recommendation                                                     | Status          |
|---------|----------------|-------------------------------------------------------------------|------|--------------------------------------------------------------------|-----------------|
| SG-001  | Security Group | SSH exposed to 0.0.0.0/0 in the controlled initial configuration. | High | Restrict SSH to a trusted IP range or use a managed access method. | To be validated |
| IAM-001 | IAM            | User or role has more permissions than required for its function. | High | Apply least privilege and remove unnecessary permissions.          | To be assessed  |
| IAM-002 | IAM            | MFA is not enabled for a privileged identity.                     | High | Enable MFA for privileged access.                                  | To be assessed  |
| KEY-001 | Key Pair       | Private key is                                                    | High | Protect key                                                        | To be           |

|        |    |                                                                                        |        |                                                                            |                |
|--------|----|----------------------------------------------------------------------------------------|--------|----------------------------------------------------------------------------|----------------|
|        |    | at risk of insecure storage or accidental repository upload.                           |        | material, use restrictive file permissions, and exclude keys from Git.     | validated      |
| S3-001 | S3 | Public access controls are not configured according to the intended security baseline. | High   | Enable S3 Block Public Access unless public access is explicitly required. | To be assessed |
| S3-002 | S3 | Data encryption is not configured according to project requirements.                   | Medium | Enable appropriate server-side encryption.                                 | To be assessed |

## 11. SECURITY REMEDIATION AND VALIDATION

Security remediation is performed only after findings have been documented. For example, if SSH is initially exposed to the Internet, the remediation is to revoke the public rule and authorize SSH only from the trusted administrator IP address. The team should capture evidence before and after the change.

For IAM, remediation may include removing excessive policies, moving users into appropriately permissioned groups, enabling MFA, and replacing unnecessary long-lived credentials with IAM roles. For S3, remediation may include enabling Block Public Access, configuring encryption, and enabling versioning.

Validation is performed by repeating the relevant AWS CLI query and confirming the expected security state in the AWS Management Console. The result should be recorded in the findings register as Remediated or Validated.

## 12. GITHUB DOCUMENTATION AND VERSION CONTROL

GitHub is used as the central repository for project code, documentation, screenshots, architecture diagrams, and the final report. The repository should contain a README file explaining the project, Bash scripts for automation, security assessment reports, screenshots, the architecture diagram, and the final project report.

A .gitignore file is essential. Private EC2 key files, AWS credentials, environment files, and other secrets must not be committed to GitHub. This is itself an important security practice and demonstrates that the project team understands the risks associated with secret exposure.

### 13. CHALLENGES ENCOUNTERED

- Understanding AWS regional scope and ensuring that resources were created and verified in the same AWS region.
- Learning the difference between Azure Resource Groups and AWS resource organisation methods.
- Understanding AWS IAM permissions and the practical application of least privilege.
- Interpreting Security Group rules and recognising the risk associated with unrestricted SSH access.
- Understanding the relationship between S3 public access controls, bucket policies, and encryption.
- Managing EC2 private key files securely and preventing accidental repository exposure.
- Balancing automation through Bash scripts with verification through the AWS Management Console.

### 14. LESSONS LEARNED

- Cloud security is a shared responsibility between the cloud provider and the customer.
- Security should be considered during infrastructure design rather than after deployment.
- Least privilege reduces the impact of compromised identities.
- Security Groups should allow only the traffic required by the workload.
- Private keys and cloud credentials must be protected from accidental disclosure.
- S3 resources require deliberate access control because incorrect configurations can expose sensitive data.
- Automation improves consistency and reduces repetitive manual configuration.

- Security findings are more useful when supported by clear before-and-after evidence.
- GitHub repositories must be treated as public-facing assets when they are public, so secrets must never be committed.

## 15. SECURITY RECOMMENDATIONS

1. Apply the principle of least privilege to IAM users, groups, and roles.
2. Enable multi-factor authentication for privileged identities.
3. Avoid using the AWS root user for everyday administrative tasks.
4. Use IAM roles for applications and EC2 workloads instead of embedding long-lived access keys.
5. Restrict SSH access to trusted IP addresses or use managed access mechanisms where appropriate.
6. Avoid exposing database ports directly to the public Internet.
7. Enable Amazon S3 Block Public Access unless public access is explicitly required.
8. Enable server-side encryption for sensitive S3 data.
9. Enable S3 versioning where recovery from accidental deletion or modification is required.
10. Protect EC2 private key files using restrictive permissions and never commit them to GitHub.
11. Enable AWS CloudTrail for auditing API activity.
12. Consider AWS Config, Security Hub, and GuardDuty for broader security monitoring and compliance visibility.
13. Perform periodic reviews of IAM permissions, Security Groups, access keys, and storage permissions.

## 16. LIVE DEMONSTRATION PLAN

The live demonstration should last approximately 10–15 minutes. Each group member should demonstrate at least one component of the project. The demonstration should focus on the practical security lifecycle and should avoid exposing sensitive credentials.

| Demonstration Area | Suggested Activity                                                                  |
|--------------------|-------------------------------------------------------------------------------------|
| Member 1           | Show AWS authentication, VPC, subnet, and EC2 environment.                          |
| Member 2           | Run the Security Group audit and explain the before-and-after configuration.        |
| Member 3           | Demonstrate IAM review and explain least privilege and MFA.                         |
| Member 4           | Demonstrate S3 security controls, including Block Public Access and encryption.     |
| Member 5           | Show the GitHub repository, scripts, report, screenshots, and architecture diagram. |

## 17. CONCLUSION

This project demonstrates the practical application of cloud security best practices within a small AWS environment. By reviewing Security Groups, IAM, EC2 key pair management, and S3 permissions, the project addresses several important areas of cloud security. The use of Bash and AWS CLI also demonstrates how infrastructure and security review tasks can be automated.



The project's strongest feature is its structured security lifecycle. The environment is first discovered and assessed, security weaknesses are identified and documented, remediation is applied, and the resulting configuration is validated. This approach provides a repeatable method for improving cloud security and creates evidence that can be presented to technical and non-technical audiences.

The project further demonstrates that cloud security is not a one-time activity. Security controls should be continuously reviewed as cloud environments evolve. Strong identity management, restricted network access, secure storage, protected credentials, logging, monitoring, and regular assessments are essential to maintaining a resilient AWS environment.

## **18. REFERENCES**

Amazon Web Services. (n.d.). AWS Identity and Access Management (IAM) documentation. AWS Documentation.

Amazon Web Services. (n.d.). Amazon EC2 User Guide. AWS Documentation.

Amazon Web Services. (n.d.). Amazon S3 User Guide. AWS Documentation.

Amazon Web Services. (n.d.). AWS Security Best Practices. AWS Documentation.

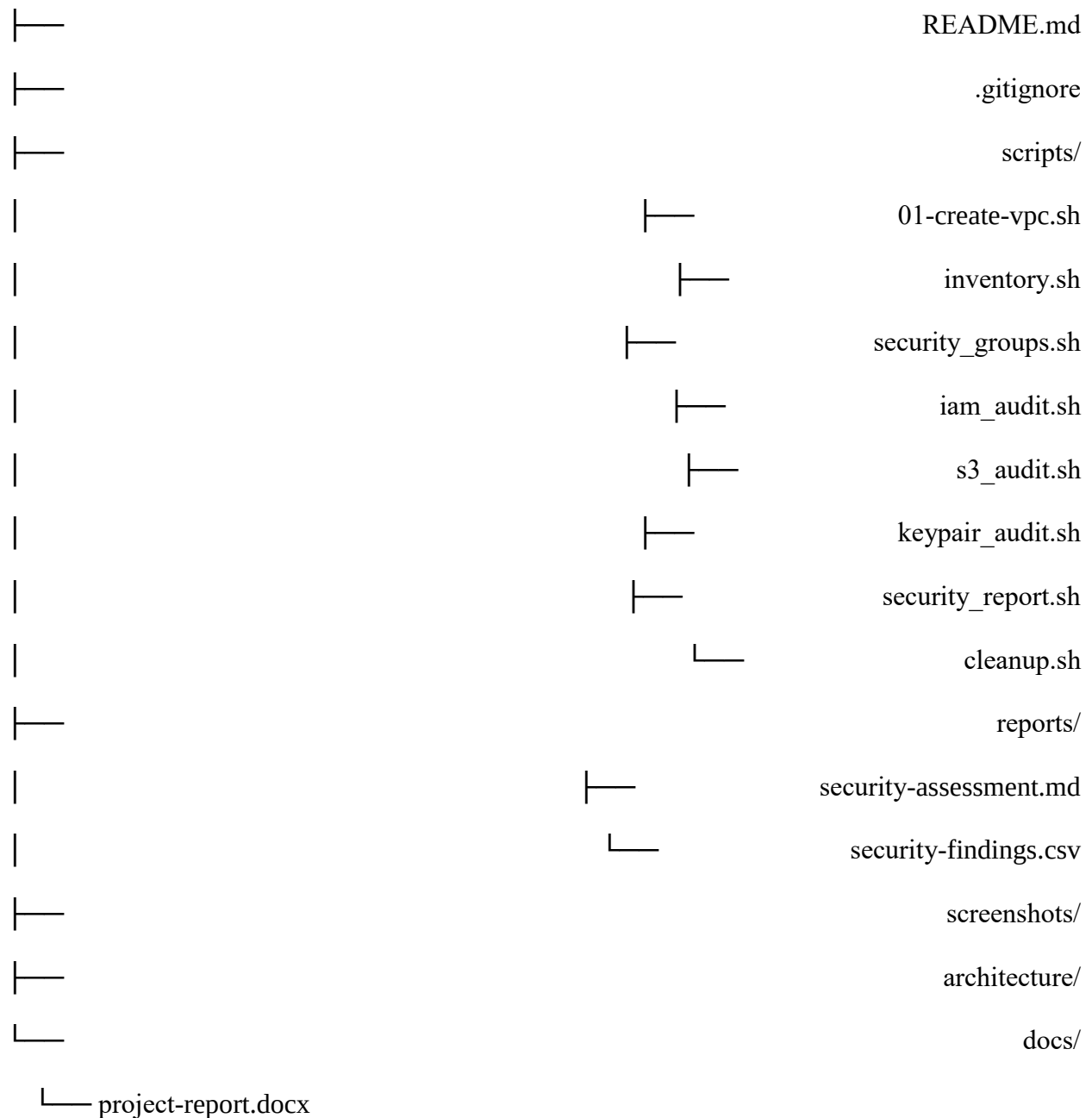
Amazon Web Services. (n.d.). AWS Well-Architected Framework: Security Pillar. AWS Documentation.

Amazon Web Services. (n.d.). AWS Command Line Interface User Guide. AWS Documentation.

## **APPENDIX A: PROJECT REPOSITORY STRUCTURE**

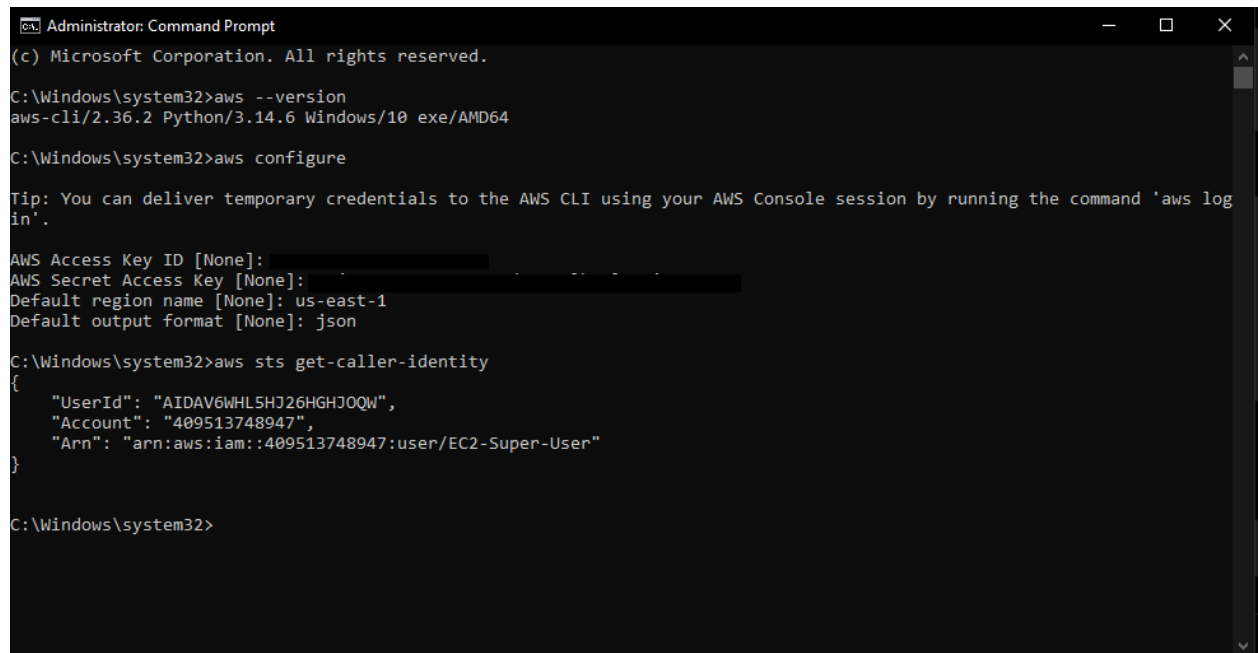
The recommended GitHub repository structure is shown below. The structure separates automation scripts, reports, screenshots, architecture documentation, and the final project report.

aws-cloud-security-project/



## APPENDIX B: SCREENSHOT PLACEHOLDERS

The following evidence should be inserted into the final report after the AWS environment is completed:



```
Administrator: Command Prompt
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>aws --version
aws-cli/2.36.2 Python/3.14.6 Windows/10 exe/AMD64

C:\Windows\system32>aws configure

Tip: You can deliver temporary credentials to the AWS CLI using your AWS Console session by running the command 'aws log
in'.

AWS Access Key ID [None]: 
AWS Secret Access Key [None]: 
Default region name [None]: us-east-1
Default output format [None]: json

C:\Windows\system32>aws sts get-caller-identity
{
  "UserId": "AIDAV6WHL5HJ26HGHJQW",
  "Account": "409513748947",
  "Arn": "arn:aws:iam::409513748947:user/EC2-Super-User"
}

C:\Windows\system32>
```

Figure 2: AWS CLI authentication using `aws sts get-caller-identity`.

```
MINGW64:/c/Users/hp/Downloads/aws-cloud-security-project/scripts
echo ""
echo "=====
echo "VPC CREATION COMPLETE"
echo "=====
=====
AWS CLOUD SECURITY PROJECT
Creating VPC
=====

Region: us-east-1
VPC CIDR: 10.0.0.0/16
VPC Name: cloud-security-vpc

VPC created successfully!
VPC ID: vpc-030d89d38b1d72ca7

VPC Name tag added.

=====
VPC CREATION COMPLETE
=====

hp@DESKTOP-186B14G MINGW64 ~/Downloads/aws-cloud-security-project/scripts
$ aws ec2 describe-vpcs --region us-east-1 --filters "Name=tag:Name,Values=cloud-security-vp
c" --query 'Vpcs[].{
    VPC_ID:VpcId,
    CIDR:CidrBlock,
    State:State
  }' --output table
-----
|                               DescribeVpcs                               |
|-----+-----+-----+-----+
|  CIDR  |  State  |  VPC_ID  |
|-----+-----+-----+-----+
| 10.0.0.0/16 | available | vpc-030d89d38b1d72ca7 |
|-----+-----+-----+-----+

hp@DESKTOP-186B14G MINGW64 ~/Downloads/aws-cloud-security-project/scripts
$ |
```

- 
- Figure 3: VPC successfully created in the eu-west-2 region.
- 
- Figure 4: Public subnet, Internet Gateway, and route table configuration.
- Figure 5: Security Group before remediation.
- Figure 6: Security Group after SSH restriction.
- Figure 7: EC2 instance running successfully.
- Figure 8: IAM users, groups, roles, or policies under review.
- Figure 9: S3 Block Public Access, encryption, and versioning configuration.

- Figure 10: GitHub repository showing scripts and documentation.
- Figure 11: Final AWS security architecture diagram.

Report preparation note: This document is a professionally structured project report draft. The group should replace the screenshot placeholders with actual evidence from the AWS environment, update the findings table to reflect the exact configurations discovered, insert the final architecture diagram, and add any institution-specific cover page information required by the lecturer.